

目次

第 1 章	はじめに	3
	本書の内容	3
	開発の背景	3
	Serial WiFi Logger	4
	対象のハードウェア	4
	ATOMS3 Lite の USB ホスト機能の制限	4
	ファームウェアの構成	4
第 2 章	ESP-IDF USB ホストドライバの構成	6
	USB Host API の概要	6
	USB ホストライブラリの初期化	7
	USB ホストライブラリのクライアントの管理	7
	USB ホストライブラリへのクライアントの登録	8
	USB デバイス検出の通知	8
	USB 転送の仕組み	8
	CDC-ACM ドライバ	9
第 3 章	FTDI ドライバの詳細実装	10
	FTDI USB シリアル変換器のプロトコル	10
	FTDI ドライバで用いるデータ構造	10
	デバイス構造体 (ftdi_dev_t)	10
	モデム状態構造体 (ftdi_modem_status_t)	11
	USB ディスクリプタの解析	11
	ベンダー固有コントロールリクエストの構築	11
	モデム制御コマンドの構築	12
	ラインプロパティの構築	12
	ボーレート除数計算	13
	モデム状態の 2 バイトヘッダー処理	13
	データの送信	15
	FTDI ドライバの処理全体	15
第 4 章	RFC2217 プロトコルの実装	17
	RFC2217 の概要	17
	Telnet プロトコル基礎	17
	オプションネゴシエーション (WILL/WONT/DO/DONT)	18
	サブネゴシエーション (IAC SB … IAC SE) の役割	18
	RFC2217 のコマンド体系	18
	RFC2217 受信データの解析処理	18
	セッション状態構造体	20

ネゴシエーションシーケンス	21
サブネゴシエーション処理	21
RFC2217 パケット生成	22
RFC2217 サーバーの構成	24
モデム状態のポーリング	25
IAC エスケープとデータ送信	25
第 5 章 シリアル制御抽象化レイヤー	27
設計思想	27
シリアル制御抽象化レイヤーの API	27
内部実装	27
第 6 章 TCP サーバー機能	29
TCP データサーバー (Port 8888)	29
TCP 制御サーバー (Port 8889)	29
バッファプール管理	30
双方向キューアーキテクチャ	30
USB デバイス状態管理	30
第 7 章 WiFi 接続設定と mDNS	31
WiFi 接続設定	31
mDNS サービスの公開	31
第 8 章 OTA アップデート機能	32
第 9 章 使用例	34
esptool.py による遠隔ファームウェア書き込み	34

第 1 章

はじめに

本書の内容

本書では、USB-CDC 準拠および FTDI 製の USB-UART 変換器を無線 LAN 接続のネットワーク経由で読み書きするための ESP32-S3 用ファームウェアの構成について解説します。ファームウェアを含む全体のシステム構成や、USB ホスト機能、ネットワーク経由でシリアルポートを扱う方法について解説します。解説するファームウェアのソースコードは GitHub にて公開しています。^{*1}

本書では ESP32 シリーズ向けの開発環境である ESP-IDF を使った USB ホスト機能を持つファームウェアの開発について解説しますが、USB の通信そのものについては解説しません。USB 通信の基礎的な内容については既知のものとします。例えば USB のディスクリプタの内容やエンドポイントごとの転送といったことは扱いません。

開発の背景

インターネットに接続して動作する IoT 機器では、中国 Espressif Systems の ESP32 シリーズをはじめとした無線機能付き MCU が多く使われます。

筆者は ESP32 シリーズを使った IoT 機器のファームウェア開発を業務で行っていますが、こういったファームウェア開発時に問題となるのが、動作時のログの収集です。

MCU のログは多くの場合は MCU の UART 通信機能をもちいて特定のピンから出力されるので、PC に USB-UART 変換器経由で接続し、ログを読み取ります。これらのログをリモートで収集するために、Raspberry Pi に複数台の FTDI 製の USB-UART 変換器を接続し、ログをアップロードするシステムを構築していました。この時に用いる USB-UART 変換器は製品のログ収集専用の機器なので、単純に USB-UART 変換を行うだけでなく、特殊な形状の治具となっています。しかし、Raspberry Pi 自体が SBC の中では安価とはいえ、それなりに高く、また設置場所や電源、Linux 上での設定が必要などの問題がありました。

一方、ESP32-S3 をはじめとする比較的新しい ESP32 シリーズの MCU は、USB シリアル変換器を内蔵しており、USB の結線を行うだけで直接 USB 経由でログを読み取ることができます従来の USB-UART 変換器であれば、ログを別の ESP32 で読み出してネットワークに投げるのは問題なく行えましたが、内蔵 USB シリアル変換器を使う前提の回路設計となっている場合は、ログを読み取る側が USB ホストとなる必要があります。

幸いなことに、ESP32-S3 には USB ホスト機能があるので、USB ホスト機能を用いて FTDI や ESP32 の内蔵 USB シリアル変換器を制御できます。そこで、ESP32-S3 を使って USB-UART をネットワーク経由で制御するファームウェアを開発しました。

^{*1} https://github.com/ciniml/serial_wifi_logger

Serial WiFi Logger

Serial WiFi Logger は、ESP32-S3 を使用した USB シリアルデバイス ↔ WiFi 間のブリッジシステムです。USB ポートに接続した CDC-ACM 準拠デバイスまたは FTDI チップ搭載デバイスを、WiFi 経由で TCP または RFC2217 プロトコルを使ってリモートから操作・ログ収集できます。

主な機能は次のとおりです。

- **CDC-ACM と FTDI 両対応:** CDC-ACM 準拠のデバイス および FTDI デバイスを自動判別して対応
- **RFC2217 サーバー:** pyserial などの標準ライブラリからシリアルポートと同様にアクセス可能
- **TCP データブリッジ:** TCP ポート 8888 でシリアルデータを双方向転送
- **TCP 制御ポート:** TCP ポート 8889 で DTR/RTS/ボーレートテキストコマンドで制御
- **mDNS によるサービス公開:** `serial-XXXXXX._serial._tcp.local` としてローカルネットワーク上から mDNS により名前解決可能
- **WiFi プロビジョニング:** Espressif の SoftAP ベースの無線 LAN 初期設定機能
- **OTA アップデート:** HTTP 経由のファームウェア更新機能

開発環境としては、Espressif が提供する ESP32 シリーズ向けの標準の開発環境である ESP-IDF を使用します。バージョンは執筆時点での最新の stable 版である ESP-IDF v6.0 です。また、対象の MCU は ESP32-S3 とします。

対象のハードウェア

ESP32-S3 を搭載した開発用モジュールとして、M5Stack の ATOMS3 Lite を使用することを前提とします。ATOMS3 Lite は執筆時点でスイッチサイエンスにて 1760 円で購入可能です。^{*2}

ATOMS3 Lite の USB ホスト機能の制限

ATOMS3 Lite は USB ホスト機能を使わない場合は、本体付属の USB Type-C ポートから電源供給やファームウェアの書き込み、デバッグログの出力を行えます。USB ホスト機能を使う場合は、本体付属の USB Type-C ポートはホスト機能で占有されるため、別途裏面のピンソケットからの電源供給が必要です。このため、裏面のピンソケットから別の USB Type-C ケーブルにて電源供給をおこなうためのアダプタ基板を作成しました。

また、デバッグ用のログ出力を行えなくなるので、ログ出力先の UART のピンを ATOMS3 Lite の GROVE 互換ポートのピンに変更し、GROVE ケーブルで別の USB-UART アダプタを接続できるようにしています。

図 図 1.1 にハードウェア構成を示します。

ファームウェアの構成

Serial WiFi Logger のファームウェアは主に次のモジュールから構成されています。

- USB ホスト機能
 - CDC-ACM ホストドライバ (Espressif のコンポーネント)
 - FTDI チップ用ドライバ
- TCP 通信機能
 - シリアル通信

^{*2} <https://www.switch-science.com/products/8778?srsId=AfmBOoqSRRpklmOigbM0OC4PJQ86iSYr5kjDNMeyeiGD8EwIAkC0JFo>

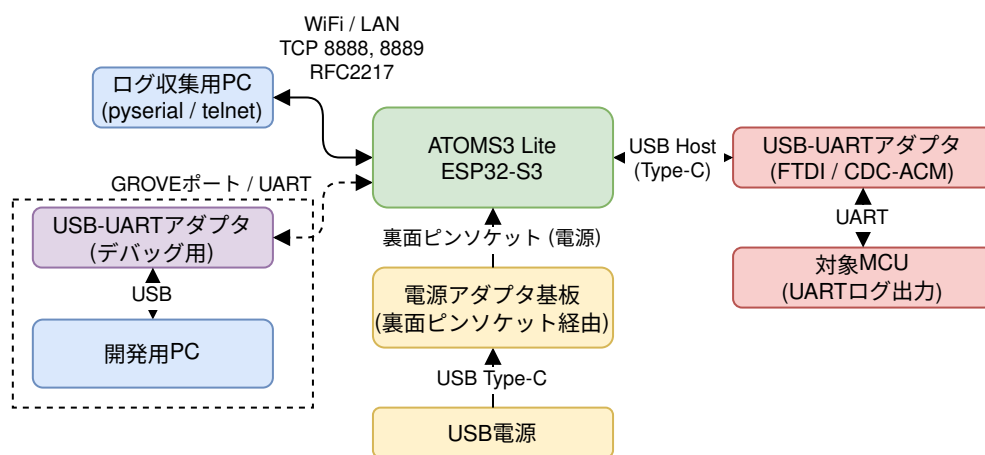


図 1.1 ハードウェア構成

- 制御機能 (DTR, RTS 制御等)
- 管理機能
 - WiFi 接続設定
 - OTA アップデート

以降の章では、これらのモジュールの実装について順に説明していきます。

第 2 章

ESP-IDF USB ホストドライバの構成

USB Host API の概要

ESP32-S3 は前述のとおり、USB OTG コントローラを搭載しているため、USB Host になることができます。USB Host 機能をつかうには、ESP-IDF の USB Host ライブラリ (`usb/usb_host.h`) を使います。USB Host ライブラリは次のような階層構造になっています。

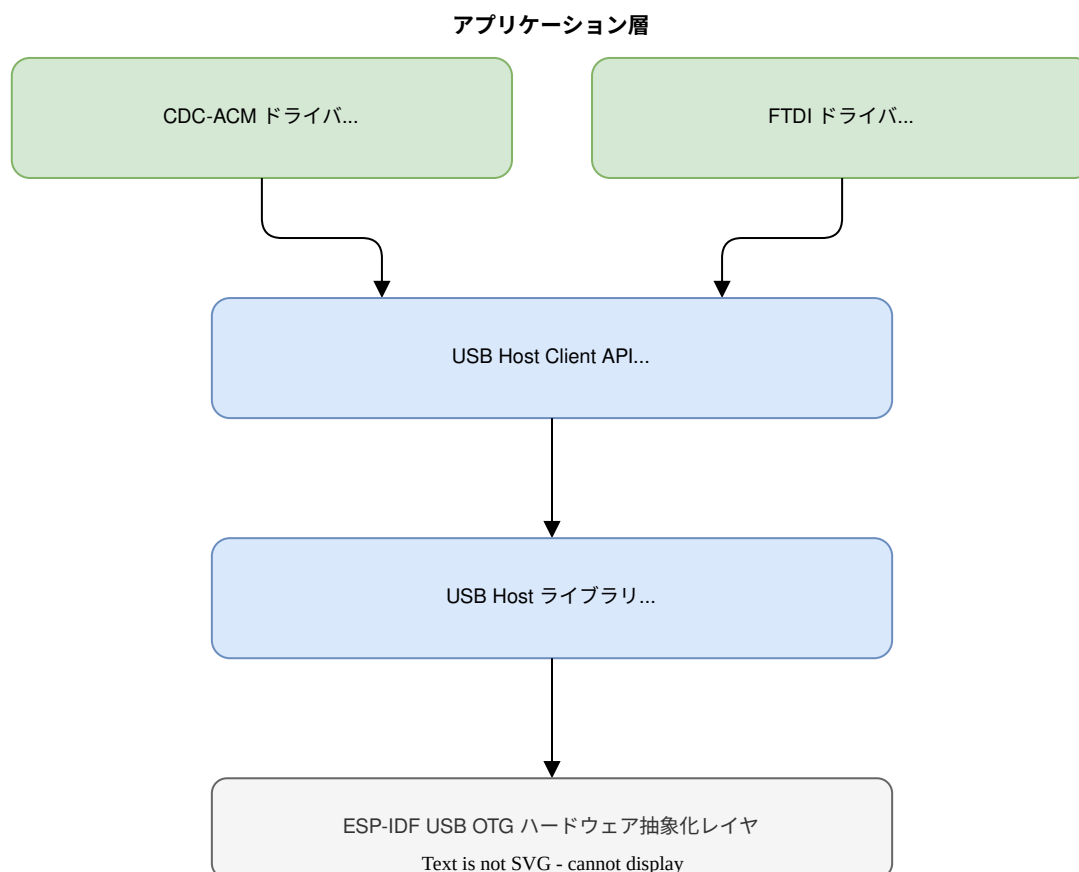


図 2.1 USB ホストライブラリ層構成

CDC-ACM ドライバと FTDI ドライバはどちらも **USB ホストクライアント** として USB Host ライブラリに登録され、それぞれが独立して USB デバイスイベントを受け取ります。アプリケーションはどちらのドライバとも直接やりとりしますが、後述する USB シリアル制御抽象化レイヤーを挟むことでドライバの違いを意識しなくて済むようにしています。

USB ホストライブラリの初期化

main.c の usb_lib_task タスクが USB ホストライブラリのイベントループを担います。

```
static void usb_lib_task(void*arg)
2 {
    // USB ホストライブラリを初期化
4     const usb_host_config_t host_config = {
        .skip_phy_setup = false,
6         .intr_flags = ESP_INTR_FLAG_LEVEL1,
    };
8     ESP_ERROR_CHECK(usb_host_install(&host_config));

10    // ドライバタスクへ通知 (CDC-ACM / FTDI)
    xTaskNotifyGive(driver_task_handle);
12
    // USB ホストライブラリのイベントを処理
14    bool has_clients = true;
    bool has_devices = true;
16    while (has_clients || has_devices) {
        uint32_t event_flags;
18        usb_host_lib_handle_events(portMAX_DELAY, &event_flags);

20        if (event_flags & USB_HOST_LIB_EVENT_FLAGS_NO_CLIENTS) {
            has_clients = false;
22        }
        if (event_flags & USB_HOST_LIB_EVENT_FLAGS_ALL_FREE) {
24            has_devices = false;
        }
26    }
    usb_host_uninstall();
28    vTaskDelete(NULL);
}
```

USB ホストライブラリの準備ができる前にクライアントを登録しようとするとエラーになるため usb_host_install() の後にドライバタスクへ通知し、ドライバの初期化を許可しています。

USB ホストライブラリのクライアントの管理

ESP-IDF の USB Host ライブラリは、複数のクライアント (ドライバ) が同時に動作できる設計になっています。各クライアントは個別のハンドル (usb_host_client_handle_t) を持ち、それぞれが独立してデバイスの接続・切断イベントを受け取ります。

FTDI ドライバは、ドライバ全体の状態を管理するシングルトン構造体 ftdi_sio_obj_t を p_ftdi_sio_obj というファイルスコープの静的ポインタで保持しています。

```
typedef struct {
2     usb_host_client_handle_t ftdi_client_hdl; // USB Host クライアントハンドル
    SemaphoreHandle_t open_close_mutex; // デバイス open/close の排他制御
4     EventGroupHandle_t event_group; // ドライバ終了シグナル用
    ftdi_sio_new_dev_callback_t new_dev_cb; // 新デバイス検出コールバック
6     void* new_dev_cb_arg;
    SLIST_HEAD(list_dev, ftdi_dev_s) ftdi_devices_list; // 接続デバイスのリスト
8 } ftdi_sio_obj_t;

10 static ftdi_sio_obj_t* p_ftdi_sio_obj = NULL;
```

p_ftdi_sio_obj は ftdi_sio_host_install() 呼び出し時にヒープ確保・初期化され、ftdi_sio_host_uninstall() で解放されて NULL に戻ります。ドライバがインストールされていない状態で API を呼び出した場合は ESP_ERR_INVALID_STATE を返すガード処理がそれぞれの関数冒頭に入っています。

`ftdi_devices_list` は BSD リストマクロ (`SLIST_`) を使った単方向リストで、接続中の `ftdi_dev_t` を管理します。デバイスのオープン時にリストへ追加され、クローズ時に削除されます。複数の FTDI デバイスを同時に接続しても、それぞれが独立したエントリとしてリストで管理されます。

USB ホストライブラリへのクライアントの登録

各ドライバは `usb_host_client_register()` で USB ホストライブラリのクライアントとして登録します。登録時にデバイスの接続・切断イベントを受け取るコールバックを渡します。FTDI ドライバでの使用例を以下に示します。

```
const usb_host_client_config_t client_config = {
2  .is_synchronous = false,
  .max_num_event_msg = 5,
4  .async = {
    .client_event_callback = usb_event_cb, // イベントコールバック
6    .callback_arg = NULL,
    },
8 };
usb_host_client_register(&client_config, &p_ftdi_sio_obj->ftdi_client_hdl);
```

USB デバイス検出の通知

`usb_event_cb` は `USB_HOST_CLIENT_EVENT_NEW_DEV` イベントで新しいデバイスの接続を知らせます。

```
static void usb_event_cb(const usb_host_client_event_msg_t *event_msg, void *arg)
2 {
  switch(event_msg->event) {
4    case USB_HOST_CLIENT_EVENT_NEW_DEV:
      // 新デバイス検出 → アプリケーションの new_dev_cb を呼び出す
6      if(p_ftdi_sio_obj->new_dev_cb) {
        uint8_t dev_addr = event_msg->new_dev.address;
8        usb_device_handle_t dev_hdl;
        usb_host_device_open(p_ftdi_sio_obj->ftdi_client_hdl, dev_addr, &dev_hdl);
10       // VID/PID を取得してコールバック
        const usb_device_desc_t *dev_desc;
12       usb_host_get_device_descriptor(dev_hdl, &dev_desc);
        p_ftdi_sio_obj->new_dev_cb(dev_desc->idVendor, dev_desc->idProduct,
14       p_ftdi_sio_obj->new_dev_cb_arg);
      }
16       break;
    }
18 }
```

`main.c` 側の `new_dev_cb` では VID/PID を見てドライバを選択します。FTDI デバイス (VID=0x0403) であれば `ftdi_sio_host_open()` を呼び、そうでなければ CDC-ACM ドライバに引き渡します。

USB 転送の仕組み

ESP-IDF の USB Host API では、USB での転送を `usb_transfer_t` 構造体で表現します。`usb_transfer_t` 構造体のフィールドに転送処理の内容と完了時に呼びだされるコールバック関数を設定し、`usb_host_transfer_submit` 関数を呼び出します。

```
// 転送バッファの確保 (DMA対応バッファ)
2 usb_host_transfer_alloc(buffer_size, 0, &transfer);

4 // 転送パラメータ設定
transfer->device_handle = dev_hdl;
6 transfer->bEndpointAddress = ep_addr; // エンドポイントアドレス
transfer->num_bytes = buffer_size;
8 transfer->callback = in_xfer_cb; // 完了コールバック
transfer->context = ftdi_dev; // コールバックに渡されるユーザー・コンテキスト
10
```



```
// 転送の送信 (非同期)
```

```
12 usb_host_transfer_submit(transfer);
```

転送が完了すると callback が呼び出されます。バルク IN 転送の場合、in_xfer_cb でデータを処理したあと、直ちに同じ転送を再サブミットして連続受信を続けます。

```
// in_xfer_cb の最後で再サブミット
```

```
2 ESP_ERROR_CHECK(usb_host_transfer_submit(transfer));
```

一方、ボーレート変更などの設定コマンドはコントロール転送で送ります。こちらはセマフォを使ったブロッキング方式で、転送完了を待ってから次の処理へ進みます。

```
usb_host_transfer_submit_control(client_hdl, ctrl_transfer);
```

```
2 xSemaphoreTake(ctrl_transfer_semaphore, pdMS_TO_TICKS(FTDI_CTRL_TIMEOUT_MS));
```

CDC-ACM ドライバ

CDC-ACM は USB の標準クラスで、ESP32 内蔵の USB シリアル変換器や多くの USB-UART 変換器がこのクラスを使っています。ESP-IDF では ESP Component Registry のコンポーネント `espressif/usb_host_cdc_acm` として CDC-ACM のドライバが提供されています。^{*1}

```
// CDC-ACM ドライバのインストール
```

```
2 cdc_acm_host_install(NULL);
```

```
4 // デバイスのオープン
```

```
cdc_acm_host_open(VID, PID, interface_num, &dev_config, &cdc_hdl);
```

```
6
```

```
// ライン設定 (ボーレート/データビット/パリティ/ストップビット)
```

```
8 cdc_acm_line_coding_t coding = {
```

```
    .dwDTERate = 115200,
```

```
10    .bDataBits = 8,
```

```
    .bParityType = 0, // None
```

```
12    .bCharFormat = 0, // 1 stop bit
```

```
};
```

```
14 cdc_acm_host_line_coding_set(cdc_hdl, &coding);
```

```
16 // モデム制御 (DTR/RTS)
```

```
cdc_acm_host_set_control_line_state(cdc_hdl, true, false); // DTR=ON, RTS=OFF
```

```
18
```

```
// データ送信
```

```
20 cdc_acm_host_data_tx_blocking(cdc_hdl, data, len, timeout_ms);
```

受信データはオープン時に登録したコールバック関数 (data_cb) で非同期に通知されます。

^{*1} https://components.espressif.com/components/espressif/usb_host_cdc_acm/

第 3 章

FTDI ドライバの詳細実装

FTDI USB シリアル変換器の Protokol

USB シリアル変換器として、FTDI 製の USB シリアル変換器が良く使われます。CDC-ACM が USB の標準クラスであるのに対し、FTDI デバイスはベンダー固有クラス (bDeviceClass=0xFF) を使用しており、コントロール転送のコマンドも独自仕様です。そのため、CDC-ACM ドライバはそのまま使えず、独自ドライバが必要になります。

本ドライバが対応している FTDI チップは次のとおりです。

チップ	PID	ベースクロック	特徴
FT232R	0x6001	3 MHz	最も一般的なシングルポート
FT232H	0x6014	12 MHz	USB2.0 HS 対応
FT2232D/H/C	0x6010	6 MHz	デュアルポート
FT4232H	0x6011	6 MHz	クアッドポート
FT230X	0x6015	3 MHz	低コスト版

チップによってベースクロックが異なり、ボーレート計算に影響します。詳細は後述のボーレート除数計算の節で説明します。

FTDI ドライバで用いるデータ構造

デバイス構造体 (ftdi_dev_t)

デバイスごとの状態は ftdi_dev_t 構造体で管理します。esp_private/ftdi_host_common.h で定義されており、ドライバ内部からのみアクセスします。

```

typedef struct ftdi_dev_s {
2  usb_device_handle_t dev_hdl; // USB デバイスハンドル
  usb_config_desc_t *config_desc; // USB コンフィグレーション記述子
4
  ftdi_chip_type_t chip_type; // チップ種別 (232R / 232H 等)
6  uint16_t vid;
  uint16_t pid;
8
  struct {
10  usb_transfer_t *in_xfer; // バルク IN 転送オブジェクト
    usb_transfer_t *out_xfer; // バルク OUT 転送オブジェクト
12  uint8_t *in_data_buffer_base; // IN バッファの先頭アドレス
    uint16_t in_mps; // IN エンドポイントの最大パケットサイズ
14  uint16_t out_mps; // OUT エンドポイントの最大パケットサイズ
    size_t in_buffer_size;
16  size_t out_buffer_size;

```

```

18  }data;
20  ftdi_modem_status_t modem_status_current; // 現在のモデム状態
21  ftdi_modem_status_t modem_status_last; // 前回のモデム状態 (変化検出用)

22  ftdi_sio_host_dev_callback_t event_cb; // イベントコールバック
23  ftdi_sio_data_callback_t data_cb; // データ受信コールバック
24  void *cb_arg;

26  SLIST_ENTRY(ftdi_dev_s) list_entry; // デバイスリストのリンク
}ftdi_dev_t;

```

モデム状態構造体 (ftdi_modem_status_t)

FTDI デバイスは、バルク IN パケットの先頭 2 バイトに常にモデム状態を付加します。モデム状態を表す `ftdi_modem_status_t` 構造体を次に示します。

```

typedef struct {
2  // バイト 0 (B0): ラインステータス
   uint8_t data_pending:1; // デバイスバッファに未読データあり
4  uint8_t overrun:1; // データオーバーランエラー
   uint8_t parity_error:1; // パリティエラー
6  uint8_t framing_error:1; // フレーミングエラー
   uint8_t break_received:1; // ブレーク信号受信
8  uint8_t tx_holding_empty:1; // 送信ホールディングレジスタ空
   uint8_t tx_empty:1; // 送信シフトレジスタ空
10 uint8_t reserved_b0:1;

12 // バイト 1 (B1): モデムステータス
   uint8_t cts:1; // Clear to Send
14 uint8_t dsr:1; // Data Set Ready
   uint8_t rts:1; // Ring Indicator
16 uint8_t rlsd:1; // Carrier Detect (RLSD/DCD)
   uint8_t reserved_b1:4;
18 }ftdi_modem_status_t;

```

USB ディスクリプタの解析

デバイスをオープンする際、まず USB 記述子を解析してバルク IN/OUT エンドポイントの情報を取り出します。USB 記述子の解析処理は `ftdi_host_descriptor_parsing.c` に実装しています。

処理の流れは次のとおりです。

`usb_host_get_active_config_descriptor()` でコンフィグレーション記述子を取得し、インターフェース記述子の中から目的のインターフェース番号を探します。見つかったらエンドポイント記述子を順に確認し、`EP_DESC_GET_EP_TYPE()` でバルク・エンドポイントかどうかをチェックします。バルク・エンドポイントであれば `EP_DESC_GET_ADDRESS()` でアドレスを、`EP_DESC_GET_MPS()` で最大パケットサイズ (MPS) を取得し、IN/OUT のエンドポイントをそれぞれ `in_ep_addr / out_ep_addr / in_mps / out_mps` に保存します。

ベンダー固有コントロールリクエストの構築

FTDI デバイスの設定変更はすべて USB のコントロール転送で行います。各リクエストは `ftdi_control_request_t` 構造体で表します。

```

typedef struct {
2  uint8_t request; // FTDI_SIO リクエストコード
   uint16_t value; // wValue フィールド
4  uint16_t index; // wIndex フィールド (インターフェース番号)
}ftdi_control_request_t;

```

FTDI 固有のリクエストコードを 表 3.2 に示します。

表 3.2: FTDI 固有のリクエスト・コード

コード	値	説明
FTDI_SIO_RESET	0	デバイス/バッファリセット
FTDI_SIO_SET_MODEM_CTRL	1	DTR/RTS 制御
FTDI_SIO_SET_FLOW_CTRL	2	フロー制御設定
FTDI_SIO_SET_BAUDRATE	3	ボーレート設定
FTDI_SIO_SET_DATA	4	データビット/パリティ/ストップビット
FTDI_SIO_GET_MODEM_STATUS	5	モデム状態取得
FTDI_SIO_SET_LATENCY_TIMER	9	レイテンシタイマー設定

モデム制御コマンドの構築

DTR/RTS の制御は FTDI_SIO_SET_MODEM_CTRL リクエストで行います。wValue の上位バイトにどの信号を制御するかのマスクを、下位バイトに信号値を格納します。

```

esp_err_t ftdi_protocol_build_set_modem_ctrl(ftdi_control_request_t* req_out,
2  bool_dtr, bool_rts)
{
4  //wValue の構造:
  //上位バイト: 制御対象のマスク
6  //下位バイト: 信号値 (1=High, 0=Low)
  //DTR は bit 0、RTS は bit 1
8  uint16_t value = 0;

10 value |= FTDI_SIO_SET_DTR_MASK << 8; //DTR の出力を変更
  if (dtr) value |= FTDI_SIO_SET_DTR_MASK;
12
  value |= FTDI_SIO_SET_RTS_MASK << 8; //RTS の出力を変更
14  if (rts) value |= FTDI_SIO_SET_RTS_MASK;

16  req_out->request = FTDI_SIO_SET_MODEM_CTRL;
  req_out->value = value;
18  req_out->index = 0;
  return ESP_OK;
20 }

```

ラインプロパティの構築

データビット・パリティ・ストップビットは FTDI_SIO_SET_DATA リクエストで設定します。データビット・パリティ・ストップビットを wValue の対応するビットに格納します。

```

//wValue ビットフィールド:
2  //bits[7:0] データビット数 (7 または 8)
  //bits[10:8] パリティ (None=0, Odd=1, Even=2, Mark=3, Space=4)
4  //bits[13:11] ストップビット (1=0, 1.5=1, 2=2)
  uint16_t value = bits | (parity << 8) | (stype << 11);
6
  req_out->request = FTDI_SIO_SET_DATA;
8  req_out->value = value;

```

ボーレート除数計算

FTDI チップのボーレート設定は、ベースクロックを整数と分数の除数で分周して行います。単純な整数による分周では特定のボーレートしか実現できないため、分数による分周の仕組みが用意されています。まずはチップの種別からボーレート計算のベースクロックを決定します。

```
switch(chip_type){
2 case_FTDI_CHIP_TYPE_232R:
  case_FTDI_CHIP_TYPE_230X:
4     base_clock=3000000; // 3MHz
    break;
6 case_FTDI_CHIP_TYPE_232H:
    base_clock=12000000; // 12MHz
8     break;
  case_FTDI_CHIP_TYPE_2232D:
10 case_FTDI_CHIP_TYPE_4232H:
    base_clock=6000000; // 6MHz
12     break;
}
```

除数は 3 ビットの分数部分を持ちます。まず $\text{base_clock} \times 8 / \text{baudrate}$ を計算して 8 倍精度の除数を求め、整数部と分数部に分解します。

```
// 8倍精度で除数を計算
2 uint32_t divisor = (base_clock * 8) / baudrate;

4 uint32_t integral_part = divisor >> 3; // 整数部
  uint32_t fractional_part = divisor & 0x07; // 小数部 (0~7)
```

分数部は対応する値を直接マップするのではなく、FTDI 独自のマッピングとなっています。分数部に対応するビット列を wValue の上位 2 ビットと wIndex にエンコードします。

```
// 分数部のビットマッピング (FTDI 仕様書より)
2 switch(fractional_part){
    case_0b000: divisor_encoded = (0b000 << 14); break; // .000
4    case_0b001: divisor_encoded = (0b011 << 14); break; // .125
    case_0b010: divisor_encoded = (0b010 << 14); break; // .250
6    case_0b011: divisor_encoded = (0b100 << 14); break; // .375
    case_0b100: divisor_encoded = (0b001 << 14); break; // .500
8    case_0b101: divisor_encoded = (0b101 << 14); break; // .625
    case_0b110: divisor_encoded = (0b110 << 14); break; // .750
10   case_0b111: divisor_encoded = (0b111 << 14); break; // .875
}
12
// エンコード済み除数の下位 16bit → wValue
14 // エンコード済み除数の上位 16bit → wIndex
  *value_out = divisor_encoded & 0xFFFF;
16 *index_out = (divisor_encoded >> 16) & 0xFFFF;
```

注意が必要なのは、分数部のビット順が線形でない点です。たとえば .125 は 0b001 ではなく 0b011、.5 は 0b100 ではなく 0b001 にマッピングされます。このようなマッピングになっている理由は不明ですが、FTDI のハードウェア仕様として、FTDI のアプリケーションノート AN_232R-01 に記載されています。

モデム状態の 2 バイトヘッダー処理

FTDI デバイスのバルク IN パケットには、すべてのパケットの先頭 2 バイトにモデム状態が付加されます。注意点として、1 回のバルク IN 転送で複数の長さ MPS のパケットが返ってくる場合があります。この場合は各パケットがそれぞれ独立した 2 バイトのモデム状態を表すヘッダーを持ちます。

バルクIN転送データ(例:MPS=64, 2パケット受信の場合):

```
2 0Offset0:[status_B0][status_B1][data_0]...[data_61] ← パケット1(64バイト)
0Offset64:[status_B0][status_B1][data_0]...[data_61] ← パケット2(64バイト)
```

そのため in_xfer_cb では、転送バッファ全体を一気に処理するのではなく、MPS 単位でパケットを切り出しながら処理します。

```
static void in_xfer_cb(usb_transfer_t *transfer)
2 {
    ftdi_dev_t *ftdi_dev = (ftdi_dev_t *)transfer->context;
    size_t total_bytes = transfer->actual_num_bytes;
    size_t offset = 0;
    const uint16_t mps = ftdi_dev->data.in_mps;

    // MPS 単位でパケットを処理
    while (offset < total_bytes) {
        size_t chunk_size = (total_bytes - offset) > mps ?
            mps : (total_bytes - offset);

        if (chunk_size >= 2) {
            // 先頭 2 バイトからモデム状態を解析
            ftdi_modem_status_t new_status;
            ftdi_protocol_parse_modem_status(
                transfer->data_buffer + offset, &new_status);

            // モデム状態が変化した場合はイベントコールバックを呼び出す
            if (memcmp(&new_status, &ftdi_dev->modem_status_current,
                sizeof(ftdi_modem_status_t)) != 0) {
                ftdi_dev->modem_status_current = new_status;
                if (ftdi_dev->event_cb) {
                    ftdi_dev->event_cb(FTDI_SIO_HOST_MODEM_STATUS,
                        ftdi_dev->cb_arg);
                }
            }

            // 先頭 2 バイトをスキップして実データをコールバックに渡す
            size_t data_len = chunk_size - 2;
            if (data_len > 0 && ftdi_dev->data_cb) {
                ftdi_dev->data_cb(transfer->data_buffer + offset + 2,
                    data_len, ftdi_dev->cb_arg);
            }
            offset += chunk_size;
        }

        // 次の受信のために転送を再サブミット(連続ポーリング)
        ESP_ERROR_CHECK(usb_host_transfer_submit(transfer));
    }
}
```

モデム状態バイトの解析は ftdi_protocol_parse_modem_status() で行います。バイト 1 のモデムステータスは上位 4 ビットに格納されている点に注意が必要です。

```
esp_err_t ftdi_protocol_parse_modem_status(const uint8_t data[2],
2 ftdi_modem_status_t *status_out)
{
    // バイト 0: ラインステータス
    status_out->data_pending = (data[0] >> 0) & 0x01;
    status_out->overrun = (data[0] >> 1) & 0x01;
    status_out->parity_error = (data[0] >> 2) & 0x01;
    status_out->framing_error = (data[0] >> 3) & 0x01;
    status_out->break_received = (data[0] >> 4) & 0x01;
    status_out->tx_holding_empty = (data[0] >> 5) & 0x01;
    status_out->tx_empty = (data[0] >> 6) & 0x01;

    // バイト 1: モデムステータス(上位 4 ビットに格納)
    status_out->cts = (data[1] >> 4) & 0x01;
```

```

    status_out->dsr = (data[1] >> 5) & 0x01;
16    status_out->ri = (data[1] >> 6) & 0x01;
    status_out->rlsd = (data[1] >> 7) & 0x01; // Carrier Detect
18
    return ESP_OK;
20 }

```

データの送信

TCP → USB 方向のデータ送信は `ftdi_sio_host_data_tx_blocking()` で行います。

```

esp_err_t ftdi_sio_host_data_tx_blocking(ftdi_sio_dev_hdl_t ftdi_hdl,
2    const uint8_t* data, uint_t data_len, uint32_t timeout_ms)
{
4    ftdi_dev_t* ftdi_dev = (ftdi_dev_t*) ftdi_hdl;

6    xSemaphoreTake(ftdi_dev->data.out_mux, portMAX_DELAY);

8    memcpy(ftdi_dev->data.out_xfer->data_buffer, data, data_len);
    ftdi_dev->data.out_xfer->num_bytes = data_len;
10   ftdi_dev->data.out_xfer->timeout_ms = timeout_ms;

12   esp_err_t err = usb_host_transfer_submit(ftdi_dev->data.out_xfer);

14   xSemaphoreGive(ftdi_dev->data.out_mux);
    return err;
16 }

```

`out_mux` セマフォで OUT 転送の多重発行を防ぎます。バルク IN とは異なり、バルク OUT は都度サブミットして完了を待ちます。

FTDI ドライバの処理全体

FTDI ドライバの初期化から終了までの流れをまとめると次のようになります。

```

ftdi_sio_host_install()
2  |
   |  ── up_ftdi_sio_obj を確保・初期化
4  |  ── usb_host_client_register() でクライアント登録
   |  ── ftdi_client_task を起動
6
   |  (デバイス接続イベント)
8  |
   |  ── usb_event_cb(USB_HOST_CLIENT_EVENT_NEW_DEV)
10 |  ── new_dev_cb(vid, pid) → アプリに通知

12 ftdi_sio_host_open(vid, pid, interface, config, &hdl)
   |
14 |  ── ftdi_dev_t を確保
   |  ── USB デバイスをオープン (usb_host_device_open)
16 |  ── USB 記述子を解析してエンドポイント情報を取得
   |  ── バルク IN/OUT 転送バッファを確保 (usb_host_transfer_alloc)
18 |  ── チップ種別を VID/PID から判定
   |  ── デバイスリセット (FTDI_SIO_RESET)
20 |  ── レイテンシタイマー設定 (FTDI_SIO_SET_LATENCY_TIMER)
   |  ── バルク IN 転送を開始 (usb_host_transfer_submit) → 連続ポーリング開始
22
   |  (データ受信)
24 |  |
   |  ── in_xfer_cb → モデム状態除去 → data_cb → アプリに通知
26
ftdi_sio_host_close(hdl)
28 |

```

```

30      |   └─┬─┐ エンドポイントを停止・フラッシュ (usb_host_endpoint_halt/flush)
        |   └─┬─┐ 転送バッファを解放 (usb_host_transfer_free)
            |   └─┬─┐ USB デバイスをクローズ (usb_host_device_close)
32          |       |
          ftdi_sio_host_uninstall()
34          |
          |   └─┬─┐ クライアントを登録解除 (usb_host_client_deregister)
              |   └─┬─┐

```


第 4 章

RFC2217 プロトコルの実装

RFC2217 の概要

RFC2217^{*1} は 1996 年に策定された Telnet 拡張プロトコルで、TCP 接続上でリモートのシリアルポートを制御できます。Telnet オプション番号 44 (TELNET_COM_PORT_OPTION) を使ってリモートのシリアルポートのボーレート・データビット・DTR/RTSなどを制御し、シリアルポートに流すデータは Telnet の IAC エスケープ処理を施したうえでそのまま流します。

このプロトコルを採用した大きな理由は、Python の `pyserial` ライブラリが対応しているからです。つまり、`pyserial` を用いている `esptool.py` などの既存のシリアル通信コードのポート名を変えるだけで、ほぼそのままリモート対応にできます。

```
import serial
2 # RFC2217 経由でシリアルポートにアクセス
  ser = serial.serial_for_url("rfc2217://192.168.1.100:2217", baudrate=115200)
4 ser.write(b"Hello")
  data = ser.read(10)
```

Telnet プロトコル基礎

RFC2217 は Telnet の IAC (Interpret As Command, 0xFF) の仕組みを用います。0xFF に続くバイトがコマンドを表し、コマンドに続くバイトがオプション番号やパラメータを表します。

バイト値	定数	意味
0xFF	TELNET_IAC	コマンドの開始
0xFB	TELNET_WILL	オプション有効化を申告
0xFC	TELNET_WONT	オプション無効化を申告
0xFD	TELNET_DO	オプション有効化を要求
0xFE	TELNET_DONT	オプション無効化を要求
0xFA	TELNET_SB	サブネゴシエーション開始
0xF0	TELNET_SE	サブネゴシエーション終了

送信するデータ列中に 0xFF バイトが出現した場合は、0xFF0xFF とエスケープして IAC と区別します。

^{*1} <https://www.rfc-editor.org/rfc/rfc2217.html>

オプションネゴシエーション (WILL/WONT/DO/DONT)

Telnet のオプションは「使う/使わない」を双方向で合意してから有効になります。4 種類のコマンドを組み合わせで交渉します。

コマンド	意味
IAC WILL X	「私はオプション X を有効にする」と申告
IAC WONT X	「私はオプション X を有効にしない」と申告
IAC DO X	「あなたはオプション X を有効にしてほしい」と要求
IAC DONT X	「あなたはオプション X を無効にしてほしい」と要求

RFC2217 では接続直後にサーバーが WILL_COM-PORT-OPTION と DO_COM-PORT-OPTION を送り、クライアントが DO_COM-PORT-OPTION と WILL_COM-PORT-OPTION で応答することで COM ポート制御オプションが有効になります。本実装ではさらに、8 ビットデータを正しく転送するための BINARY オプション (RFC 856) と、Go Ahead シグナルを省略する SGA オプション (RFC 858) も同時にネゴシエートします。

サブネゴシエーション (IAC SB … IAC SE) の役割

オプションが有効になった後、具体的なパラメータ (ボーレートの値など) を交換するしくみがサブネゴシエーションです。IAC_SB[オプション番号][データ]IAC_SE の形式で送受信します。RFC2217 の場合、SET_BAUDRATE や SET_DATASIZE といったコマンドがこの仕組みで運ばれます。

RFC2217 のコマンド体系

RFC2217 のコマンドは、クライアントからサーバーに送るコマンドが 0~12、サーバーからクライアントへの応答がコマンドに対して 100 を加算した値で定義されています。

コマンド値	定数	応答値	説明
0	RFC2217_SIGNATURE	100	サーバー識別文字列の交換
1	RFC2217_SET_BAUDRATE	101	ボーレート設定 (4 バイト big-endian)
2	RFC2217_SET_DATASIZE	102	データビット数 (5~8)
3	RFC2217_SET_PARITY	103	パリティ設定
4	RFC2217_SET_STOPSIZE	104	ストップビット数
5	RFC2217_SET_CONTROL	105	DTR/RTS/ブレーク/フロー制御
6	RFC2217_NOTIFY_LINSTATE	106	ライン状態通知
7	RFC2217_NOTIFY_MODEMSTATE	107	モデム状態通知 (CTS/DSR/RI/CD)
10	RFC2217_SET_LINSTATE_MASK	110	ライン状態通知マスク
11	RFC2217_SET_MODEMSTATE_MASK	111	モデム状態通知マスク
12	RFC2217_PURGE_DATA	112	バッファのページ

RFC2217 受信データの解析処理

Telnet のコマンドシーケンスは複数バイトにまたがるため、1 バイトごと処理をし状態を更新して解析します。

```

typedef enum {
2  RFC2217_STATE_DATA, // 通常データモード
  RFC2217_STATE_IAC, // IAC(0xFF)を受信済み
4  RFC2217_STATE_WILL, // IAC_WILLを受信済み(次のバイトはオプション番号)
  RFC2217_STATE_WONT, // IAC_WONTを受信済み
6  RFC2217_STATE_DO, // IAC_DOを受信済み
  RFC2217_STATE_DONT, // IAC_DONTを受信済み
8  RFC2217_STATE_SB, // IAC_SBを受信済み(サブネゴシエーション開始)
  RFC2217_STATE_SB_DATA, // サブネゴシエーションデータ収集中
10 RFC2217_STATE_SB_IAC, // サブネゴシエーション中に IACを受信
} rfc2217_parser_state_t;

```

状態遷移を図にすると次のようになります。

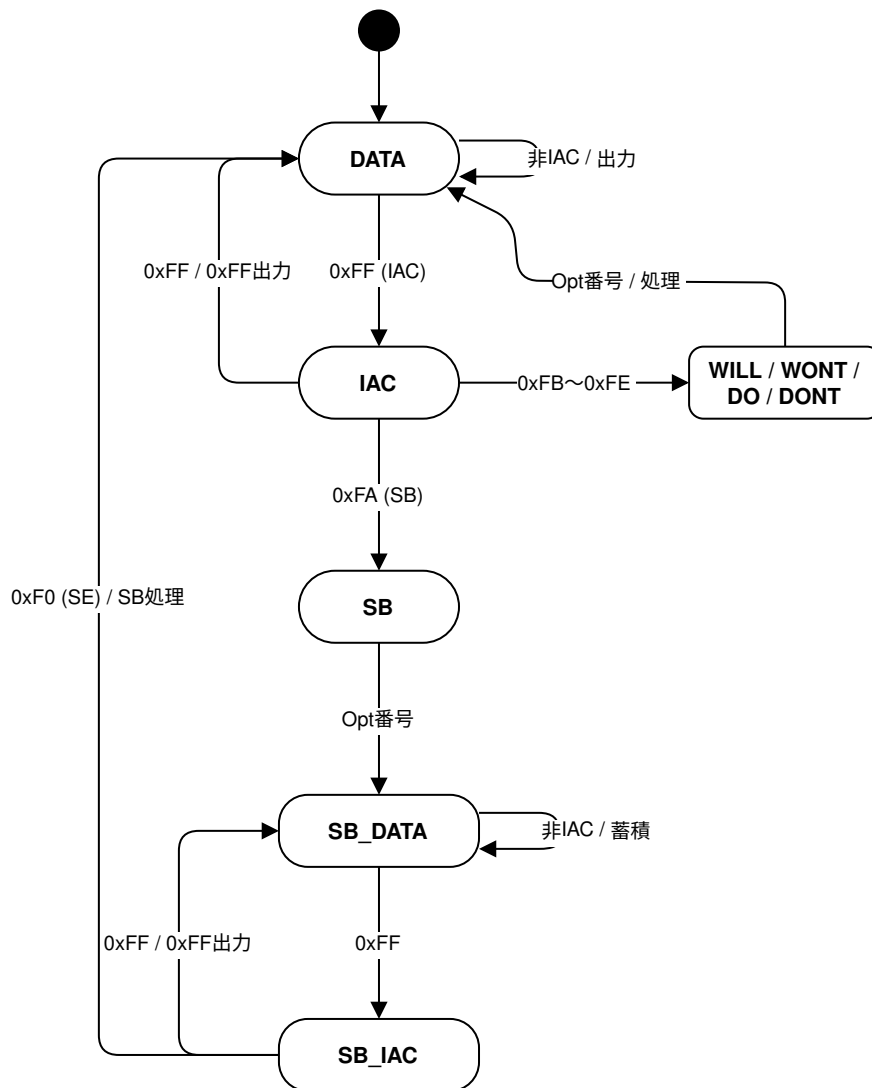


図 4.1 RFC2217 パーサーの状態遷移

コアとなる `rfc2217_parse_byte()` の実装を抜粋します。DATA 状態で `TELNET_IAC != 0xFF` 以外のバイトを受信した場合そのままデータとして出力します。0xFF を受信した場合は IAC 状態へ遷移します。IAC 状態で再度 0xFF を受け取った場合はエスケープされた 0xFF として出力します。

```

rfc2217_result_t rfc2217_parse_byte(rfc2217_session_t *session, uint8_t byte,
2  uint8_t *out_data, size_t *out_len)
{
4  *out_len = 0;

6  switch (session->state) {
    case RFC2217_STATE_DATA:
8      if (byte == TELNET_IAC) {
          session->state = RFC2217_STATE_IAC;

```

```

10  return RFC2217_RESULT_CONTINUE;
    }
12  // 通常データ: そのまま出力
    out_data[0] = byte;
14  *out_len = 1;
    return RFC2217_RESULT_DATA;
16
    case RFC2217_STATE_IAC:
18  if (byte == TELNET_IAC) {
        // エスケープされた 0xFF → 1 バイトの 0xFF として出力
20  session->state = RFC2217_STATE_DATA;
        out_data[0] = TELNET_IAC;
22  *out_len = 1;
        return RFC2217_RESULT_DATA;
24  }
        // WILL/WONT/DO/DONT/SB への状態遷移
26  // ...
        }
28  }

```

セッション状態構造体

RFC2217 のセッション状態は `rfc2217_session_t` 構造体にまとめています。パーサー状態やネゴシエーション状態に加え、現在のシリアルポート設定とペンディング中のアクションフラグも保持します。

```

typedef struct {
2  rfc2217_parser_state_t state; // パーサー状態

4  // サブネゴシエーションバッファ (最大64バイト)
    uint8_t sb_buffer[RFC2217_SB_BUFFER_SIZE];
6  size_t sb_len;

8  // オプションネゴシエーション状態
    bool com_port_enabled;
10  bool client_will_com_port;
    bool client_do_com_port;
12

    // 現在のシリアルポート設定
14  uint32_t baudrate; // デフォルト: 115200
    uint8_t datasize; // RFC2217_DATASIZE_* (5~8)
16  uint8_t parity; // RFC2217_PARITY_*
    uint8_t stopsize; // RFC2217_STOPSIZE_*
18  uint8_t flowcontrol; // RFC2217_CONTROL_FLOW_*
    bool dtr, rts, break_state;
20

    // 通知マスク
22  uint8_t linestate_mask; // デフォルト: 0xFF (全ビット通知)
    uint8_t modemstate_mask; // デフォルト: 0xFF
24

    // 保留中のアクションフラグ
26  bool settings_changed; // いずれかの設定が変更された
    bool line_coding_changed; // ボーレート/データビット等が変更された
28  bool modem_control_changed; // DTR/RTS が変更された
    bool break_changed; // ブレーク状態が変更された
30  bool send_modemstate;
    bool send_linestate;
32

    // レスポンス生成用
34  uint8_t last_command;
    uint8_t last_control_value;
36  bool need_signature_response;
    bool need_option_response;
38  uint8_t option_response_cmd;
    uint8_t option_response_opt;
40

    char signature[RFC2217_SIGNATURE_SIZE]; // "ESP32-S3SerialWiFiLogger"

```

```
42 }_rfc2217_session_t;
```

設定変更時は即座にシリアルポートへ反映するのではなく、`settings_changed` などのフラグを立てるにとどめます。サーバータスクがフラグを確認してシリアル制御 API を呼び出す、という 2 フェーズ構成になっており、設定の適用タイミングをコントロールできます。

ネゴシエーションシーケンス

TCP 接続が確立すると、サーバーからすぐにネゴシエーションを開始します。pyserial との実際のシーケンスは次のようになります。

サブネゴシエーション処理

サブネゴシエーション (`IAC_SB...`..`IAC_SE`) を受け取ると、`process_subnegotiation()` が `sb_buffer` に蓄積されたデータを解析してセッション状態を更新します。

例としてボーレートを設定する `SET_BAUDRATE` の処理は次のようになります。ボーレートは 4 バイトの big-endian で渡されます。値が 0 の場合は現在値の問い合わせという意味なので変更しません。

```
case _RFC2217_SET_BAUDRATE:
2  if (value_len >= 4) {
    uint32_t baudrate = ((uint32_t) value[0] << 24) |
4  ((uint32_t) value[1] << 16) |
    ((uint32_t) value[2] << 8) |
6  ((uint32_t) value[3]);
    if (baudrate == 0) {
8  // 0 = 現在の設定を問い合わせ (変更なし)
        } else {
10     session->baudrate = baudrate;
        session->settings_changed = true;
12     session->line_coding_changed = true;
        }
14 }
    break;
```

`SET_CONTROL` は DTR・RTS・ブレーク・フロー制御をまとめて扱います。値によって意味が変わるため、switch 文で各ケースを処理します。

`SET-CONTROL` の値と意味を以下にまとめます。

値	定数	説明
0	<code>RFC2217_CONTROL_FLOW_REQUEST</code>	現在のフロー制御設定を問い合わせ
1	<code>RFC2217_CONTROL_FLOW_NONE</code>	フロー制御なし
2	<code>RFC2217_CONTROL_FLOW_XONXOFF</code>	XON/XOFF フロー制御
3	<code>RFC2217_CONTROL_FLOW_HARDWARE</code>	ハードウェア (RTS/CTS) フロー制御
4	<code>RFC2217_CONTROL_BREAK_REQUEST</code>	現在のブレーク状態を問い合わせ
5	<code>RFC2217_CONTROL_BREAK_ON</code>	ブレーク信号 ON

値	定数	説明
6	RFC2217_CONTROL_BREAK_OFF	ブレイク信号 OFF
7	RFC2217_CONTROL_DTR_REQUEST	現在の DTR 状態を問い合わせ
8	RFC2217_CONTROL_DTR_ON	DTR ON
9	RFC2217_CONTROL_DTR_OFF	DTR OFF
10	RFC2217_CONTROL_RTS_REQUEST	現在の RTS 状態を問い合わせ
11	RFC2217_CONTROL_RTS_ON	RTS ON
12	RFC2217_CONTROL_RTS_OFF	RTS OFF
13	RFC2217_CONTROL_INFLOW_REQUEST	現在の受信フロー制御設定を問い合わせ
14	RFC2217_CONTROL_INFLOW_NONE	受信フロー制御なし
15	RFC2217_CONTROL_INFLOW_XONXOFF	受信 XON/XOFF
16	RFC2217_CONTROL_INFLOW_HARDWARE	受信ハードウェアフロー制御

```

case RFC2217_CONTROL_DTR_ON:
2  session->dtr = true;
  session->settings_changed = true;
4  session->modem_control_changed = true;
  break;
6
case RFC2217_CONTROL_BREAK_ON:
8  session->break_state = true;
  session->settings_changed = true;
10 session->break_changed = true;
  break;

```

RFC2217 パケット生成

サーバー (ESP32 側) からクライアントへのレスポンスは `rfc2217_build` で始まる関数で生成します。すべてのサブネゴシエーションパケットは `IAC_SB_COM-PORT-OPTION[cmd][data...]` の形式となっており、`rfc2217_build_subneg()` を用いてパケットを組み立てます。データ部に `0xFF` が含まれる場合はエスケープ処理も行います。

```

size_t rfc2217_build_subneg(uint8_t cmd, const uint8_t* value,
2  size_t value_len, uint8_t* out)
{
4  size_t pos = 0;
  out[pos++] = TELNET_IAC;
6  out[pos++] = TELNET_SB;
  out[pos++] = TELNET_COM_PORT_OPTION;
8  out[pos++] = cmd;

10 // データ部 (IAC はエスケープ)
  for (size_t i = 0; i < value_len; i++) {
12  if (value[i] == TELNET_IAC) {
    out[pos++] = TELNET_IAC;
14  }
    out[pos++] = value[i];
16  }
}

```

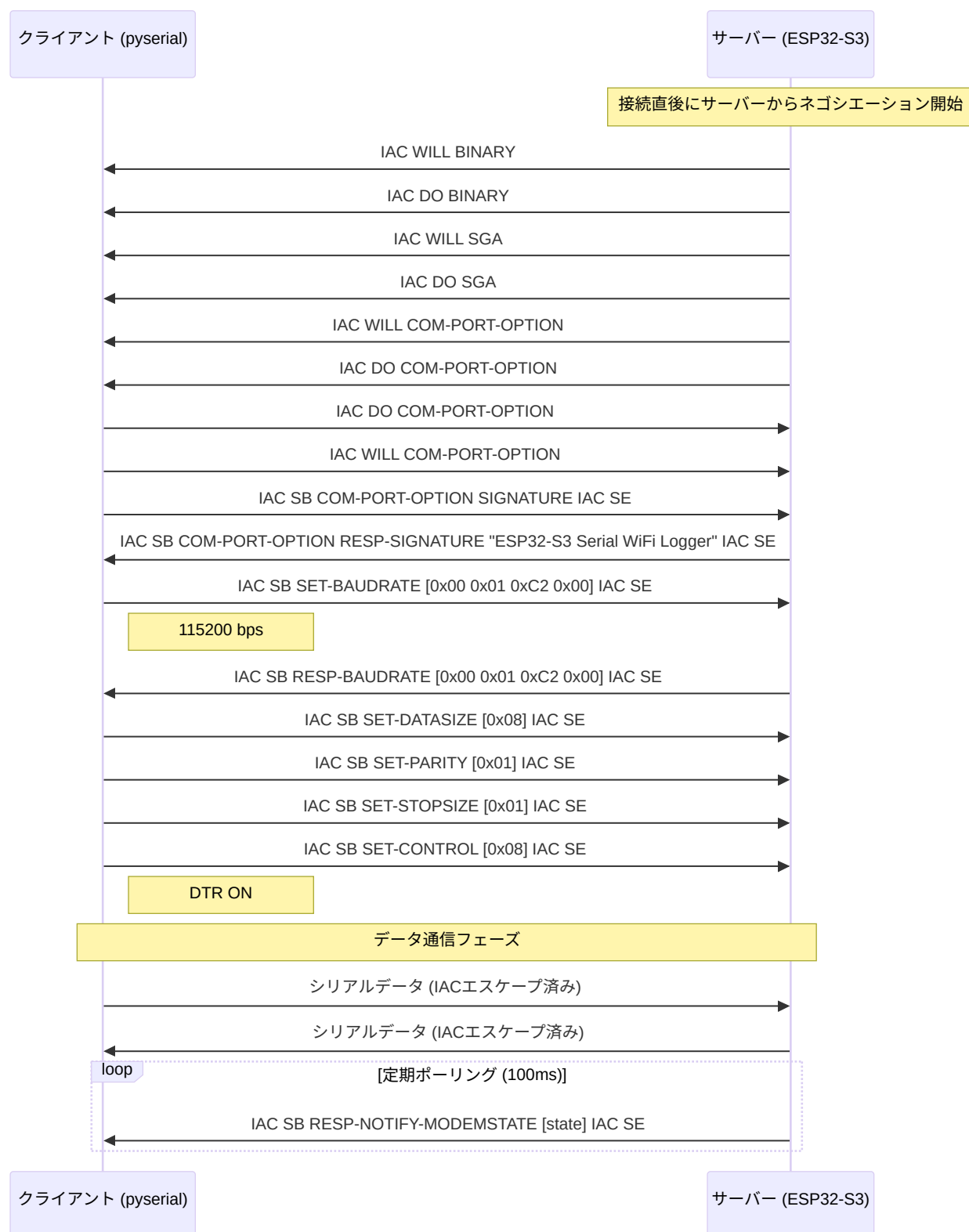


図 4.2 RFC2217 ネゴシエーションシーケンス

```

18  uint8_t out[pos++] = TELNET_IAC;
    uint8_t out[pos++] = TELNET_SE;
20  return pos;
}

```

ボーレートの応答は 4 バイト big-endian、モデム状態通知は 1 バイトで、それぞれ `rfc2217_build_subneg()` を呼んでパケットを生成します。

```

// ボーレート 応答 (4 バイト big-endian)
2  size_t rfc2217_build_baudrate_response(uint32_t baudrate, uint8_t *out)
{
4  uint8_t value[4];
    value[0] = (baudrate >> 24) & 0xFF;
6  value[1] = (baudrate >> 16) & 0xFF;
    value[2] = (baudrate >> 8) & 0xFF;
8  value[3] = baudrate & 0xFF;
    return rfc2217_build_subneg(RFC2217_RESP_SET_BAUDRATE, value, 4, out);
10 }

12 // モデム 状態 通知 (1 バイト)
    // bit 7: CD, bit 6: RI, bit 5: DSR, bit 4: CTS
14 // bit 3: Delta CD, bit 2: Delta RI, bit 1: Delta DSR, bit 0: Delta CTS
    size_t rfc2217_build_modemstate(uint8_t state, uint8_t *out)
16 {
    return rfc2217_build_subneg(RFC2217_RESP_NOTIFY_MODEMSTATE, &state, 1, out);
18 }

```

RFC2217 サーバーの構成

`rfc2217_server.c` の `rfc2217_server_task` がサーバーの本体です。SO_REUSEADDR を設定したリスニングソケットをポート 2217 で作成し、`accept()` でクライアントの接続を待ちます。同時接続は 1 クライアントのみです。

接続が来たら初期ネゴシエーション (WILL_BINARY / DO_BINARY / WILL_SGA / DO_SGA / WILL_COM-PORT-OPTION / DO_COM-PORT-OPTION) を送信し、受信ループへ入ります。受信ループでは `recv()` で取り込んだバイトを 1 バイトずつ `rfc2217_parse_byte()` に投入します。

- RFC2217_RESULT_DATA: バイトを `tx_batch` バッファに蓄積する
- RFC2217_RESULT_COMMAND: `tx_batch` に溜まったデータを `serial_control_transmit()` でフラッシュしてからシリアル設定を適用し、レスポンスを送信する

RFC2217 サーバーは TCP/USB ブリッジとは別経路でデータを送受信します。クライアントから受け取ったシリアルデータは `tcp_to_usb_queue` を経由せず、直接 `serial_control_transmit()` で USB デバイスへ送ります。逆方向 (USB → クライアント) は `rfc2217_server_send_data()` 関数経由で `send()` を呼び出します。

コマンド受信後の設定適用と応答送信は `apply_serial_settings()` と `send_response()` が担います。

```

case RFC2217_RESULT_COMMAND:
2  // tx_batch に溜まったデータをまずフラッシュ
    if (tx_batch_len > 0) {
4  serial_control_transmit(tx_batch, tx_batch_len, 1000);
        tx_batch_len = 0;
6  }
    // シリアル設定を適用
8  if (s_server.session.settings_changed) {
        apply_serial_settings(&s_server.session);
10  s_server.session.settings_changed = false;
    }
12  // レスポンスをクライアントへ送信
    send_response(sock, &s_server.session);
14  break;

```


`apply_serial_settings()` 内で各フラグを確認してシリアル制御 API を呼び出します。

```
// apply_serial_settings() の内部
2 if(session->line_coding_changed){
    serial_line_coding_t coding = {
4         .baudrate = session->baudrate,
        .data_bits = session->datasize,
6         .parity = rfc2217_parity_to_serial(session->parity),
        .stop_bits = rfc2217_stopsize_to_serial(session->stopsize),
8     };
    serial_control_set_line_coding(&coding);
10 }
    if(session->modem_control_changed){
12         serial_control_set_modem_control(session->dtr, session->rts);
    }
14 if(session->break_changed){
    serial_control_set_break(session->break_state);
16 }
```

モデム状態のポーリング

`rfc2217_modem_poll_task` は、FTDI デバイス接続時のモデム状態 (CTS/DSR/RI/CD) をポーリングして RFC2217 クライアントへ通知するためのタスクです。

CDC-ACM デバイスはモデム状態の変化を割り込みエンドポイントから自発的に通知しますが、FTDI デバイスはそのような仕組みがなく、FTDI_SIO_GET_MODEM_STATUS コントロールリクエストで明示的に読み出す必要があります。そのため、別タスクで定期的にポーリングしています。

```
static void rfc2217_modem_poll_task(void *arg)
2 {
    while(rfc2217_server_running){
4         if(rfc2217_server_is_connected()){
            serial_modem_status_t status;
6             if(serial_control_get_modem_status(&status) == ESP_OK){
                // 状態が変化した場合のみ通知
8                 if(memcmp(&status, &last_status, sizeof(status)) != 0){
                    last_status = status;
10                    uint8_t state = 0;
                    if(status.cts) state |= RFC2217_MODEMSTATE_CTS;
12                    if(status.dsr) state |= RFC2217_MODEMSTATE_DSR;
                    if(status.ri) state |= RFC2217_MODEMSTATE_RI;
14                    if(status.cd) state |= RFC2217_MODEMSTATE_CD;
                    rfc2217_server_notify_modemstate(status.cts, status.dsr,
16                    status.ri, status.cd);
                }
18            }
        }
20        vTaskDelay(pdMS_TO_TICKS(CONFIG_RFC2217_MODEM_POLL_INTERVAL_MS));
    }
22    vTaskDelete(NULL);
}
```

ポーリング間隔はデフォルト 100ms で、`menuconfig` の `RFC2217_MODEM_POLL_INTERVAL_MS` で変更できます。

IAC エスケープとデータ送信

USB から受信したデータを RFC2217 クライアントへ送る際は、`0xFF` バイトをエスケープしてから送信します。

```
size_t rfc2217_escape_data(const uint8_t *in, size_t in_len,
2 uint8_t *out, size_t out_size)
{
4     size_t j = 0;
    for(size_t i = 0; i < in_len; i++){
        if(in[i] == 0xFF){
            out[j++] = 0x00;
            out[j++] = 0xFF;
        } else {
            out[j++] = in[i];
        }
    }
    return j;
}
```

```
6  if(in[i]==TELNET_IAC){
    //0xFF→0xFF0xFF(2バイトに拡張)
8  out[j++] = TELNET_IAC;
    if(j<out_size) out[j++] = TELNET_IAC;
10 } else{
    out[j++] = in[i];
12 }
    }
14 return j;
}
```

最悪の場合、すべてのバイトが 0xFF であれば出力は入力 of 2 倍になります。出力バッファは入力 of 2 倍以上のサイズが必要です。

第 5 章

シリアル制御抽象化レイヤー

設計思想

RFC2217 サーバーやコントロールポートのコードが CDC-ACM と FTDI の両方を意識するのは煩雑です。そこで `serial_control.h` / `serial_control.c` が抽象化レイヤーとして両者の API の違いを吸収します。これによりデバイス種別を気にせず、統一した API でシリアルポートを操作できます。

シリアル制御抽象化レイヤーの API

シリアル制御抽象化レイヤーの API を以下に示します。

```

bool serial_control_is_connected(void);
2
// ライン設定 (ボーレート/データビット/パリティ/ストップビット)
4 esp_err_t serial_control_set_line_coding(const serial_line_coding_t *coding);
  esp_err_t serial_control_get_line_coding(serial_line_coding_t *coding);
6 esp_err_t serial_control_set_baudrate(uint32_t baudrate);
  esp_err_t serial_control_get_baudrate(uint32_t *baudrate);
8
// DTR/RTS 制御
10 esp_err_t serial_control_set_dtr(bool state);
   esp_err_t serial_control_set_rts(bool state);
12 esp_err_t serial_control_set_modem_control(bool dtr, bool rts);

14 // モデム状態読み出し
   esp_err_t serial_control_get_modem_status(serial_modem_status_t *status);
16
// ブレーク信号制御
18 esp_err_t serial_control_set_break(bool on);

20 // データ送信
   esp_err_t serial_control_transmit(const uint8_t *data, size_t len,
22   uint32_t timeout_ms);

```

内部実装

各関数の内部では `current_device->type` で CDC-ACM か FTDI かを判別し、それぞれのドライバ API を呼び出します。例として UART のボーレートやフォーマットを設定する `serial_control_set_line_coding` の実装を示します。

```

esp_err_t serial_control_set_line_coding(const serial_line_coding_t *coding)
2 {
   if (!serial_control_is_connected()) return ESP_ERR_INVALID_STATE;
4
   xSemaphoreTake(device_mutex, portMAX_DELAY);
6   esp_err_t ret;

```

```

8  if(current_device->type==DEVICE_TYPE_CDC){
    cdc_acm_line_coding_t cdc_coding={
10  .dwDTERate=coding->baudrate,
    .bDataBits=coding->data_bits,
12  .bParityType=coding->parity,
    .bCharFormat=coding->stop_bits,
14  };
    ret=cdc_acm_host_line_coding_set(
16  current_device->handle.cdc_hdl,&cdc_coding);

18  }else if(current_device->type==DEVICE_TYPE_FTDI){
    ret=ftdi_sio_host_set_baudrate(
20  current_device->handle.ftdi_hdl,coding->baudrate);
    if(ret==ESP_OK){
22  ret=ftdi_sio_host_set_line_property(
    current_device->handle.ftdi_hdl,
24  coding->data_bits,
    coding->stop_bits,
26  coding->parity);
    }
28  }

30  xSemaphoreGive(device_mutex);
    return ret;
32  }

```

デバイスがビジー状態で ESP_ERR_NOT_FINISHED を返した場合は、SERIAL_CTRL_RETRY_COUNT 回まで SERIAL_CTRL_RETRY_INTERVAL_MS ms 間隔でリトライします。

第 6 章

TCP サーバー機能

TCP データサーバー (Port 8888)

tcp_server_task がポート 8888 でリスニングソケットを作成し、TCP クライアントからの接続を待ちます。同時接続は 1 クライアントのみで、新しいクライアントが接続してきた場合は既存の接続をクローズしてから受け入れます。

クライアントから受信したデータはバッファプールに格納したうえで tcp_to_usb_queue に積み、tcp_usb_bridge_task が USB デバイスへ転送します。クライアント接続中は mDNS の TXT レコードの state フィールドを "open" に更新し、切断時に "disconnected" へ戻すことで、クライアント側がデバイスの状態をリモートから確認できます。

```
// 受信データをバッファプールに入れてキューに積む
2 data_buffer_t*buf=buffer_alloc();
  memcpy(buf->data,rx_buffer,len);
4 buf->len=len;
  xQueueSend(tcp_to_usb_queue,&buf,0);
```

USB から受信したデータは usb_to_tcp_bridge_task が usb_to_tcp_queue から取り出して send() でクライアントへ転送します。同一データを RFC2217 クライアントへも同時に配信するため、rfc2217_server_send_data() も呼び出します。

TCP 制御サーバー (Port 8889)

control_server_task はポート 8889 でテキストコマンドを受け付け、USB シリアルデバイスの DTR・RTS 信号やボーレートをリモートから制御できます。RFC2217 サーバーと異なり、Telnet プロトコルは使わず、単純な改行区切りのテキストコマンドを使用します。

コマンドの書式と動作を次に示します。

コマンド	書式	説明
DTR	DTR 0 または DTR 1	DTR 信号の Low/High 設定
RTS	RTS 0 または RTS 1	RTS 信号の Low/High 設定
BAUD	BAUD	ボーレートの変更 (300~921600)
VERSION	VERSION	ファームウェアバージョンの取得

コマンドが成功した場合は OK\n、失敗した場合は ERROR\n をレスポンスとして返します。VERSION コマンドのみ、OK\n の代わりにバージョン文字列 (例: v1.0.0\n) を返します。

```
$ nc 192.168.1.100 8889
2 DTR 1
  OK
4 BAUD 9600
  OK
```

```
6 VERSION
  v1.2.0
```

バッファプール管理

ヒープの断片化を防ぐため、起動時に 512 バイトバッファを DATA_BUFFER_POOL_SIZE 個 (デフォルト 64 個) まとめて確保し、静的プールとして使い回しています。

```
typedef struct {
2  uint8_t data[512]; // データバッファ
  uint_t len; // 有効データ長
4  bool in_use; // 使用中フラグ
} data_buffer_t;

6
typedef struct {
8  data_buffer_t buffers[CONFIG_DATA_BUFFER_POOL_SIZE];
  SemaphoreHandle_t mutex;
10 } buffer_pool_t;
```

バッファの確保は `in_use == false` のバッファを線形探索して返します。プールが枯渇した場合は NULL を返してデータをドロップし、警告ログを出力します。

双方向キューアーキテクチャ

USB ↔ TCP のデータ転送は 2 つの FreeRTOS キューで分離されています。キューに積むのはデータ本体ではなく、バッファプールから確保したポインタです。

USB → TCP (上り方向)

`usb_data_callback` でデータを受け取ったら、バッファプールからバッファを確保してデータをコピーし、ポインタを `usb_to_tcp_queue` に積みます。`usb_tcp_bridge_task` がキューからポインタを取り出し、TCP クライアントと RFC2217 クライアントの両方に配信します。RFC2217 クライアントへの送信時は IAC エスケープを行います。

TCP → USB (下り方向)

TCP または RFC2217 から受け取ったデータはバッファプールに格納し、`tcp_to_usb_queue` に積みます。`tcp_usb_bridge_task` がキューからポインタを取り出し、`serial_control_transmit()` で USB デバイスに送ります。

USB デバイス状態管理

USB デバイスの接続・切断は非同期に発生するため、ブリッジタスクが処理中に USB デバイスが切断されてもクラッシュしないよう管理が必要です。

```
typedef enum {
2  DEVICE_STATE_DETECTED, // デバイス検出済み
  DEVICE_STATE_OPENING, // オープン処理中
4  DEVICE_STATE_OPEN, // 通信可能
  DEVICE_STATE_ERROR, // エラー状態
6  DEVICE_STATE_DISCONNECTED // 切断済み
} device_state_t;
```

USB デバイスの切断イベントが来ると、ドライバのコールバックが `disconnected_sem` を Give します。これを受け取ったタスクが `current_device` を NULL に設定し、ブリッジタスクは `serial_control_is_connected()` でこの変化を検知して転送を一時停止します。

第 7 章

WiFi 接続設定と mDNS

WiFi 接続設定

初回起動時に NVS に WiFi 接続情報がない場合は、SoftAP モードで WiFi 接続設定を開始します。スマートフォンの「ESP SoftAP Prov」アプリで PROV_XXXXXX という名前のアクセスポイントに接続し、WiFi の SSID とパスワードを入力すると NVS に保存されます。セキュリティバージョン 1 (デフォルト) では PoP (Proof of Possession) コード abcd1234 による認証が必要で、意図しない第三者が認証情報を書き換えられないようになっています。

mDNS サービスの公開

WiFi 接続後は mDNS で自身をネットワーク上に公開します。

```

サービス名: serial-A02048 (MAC アドレス下3バイトを16進表記)
2 サービス型: _serial._tcp
ポート: 8888 (データポート)
4
TXT レコード:
6  version="1.0.0"
   control_port="8889"
8  rfc2217_port="2217"
   device="none"/"/CDC:0x1234:0x5678"/"/FTDI:0x0403:0x6001"
10 state="disconnected"/"/open"

```

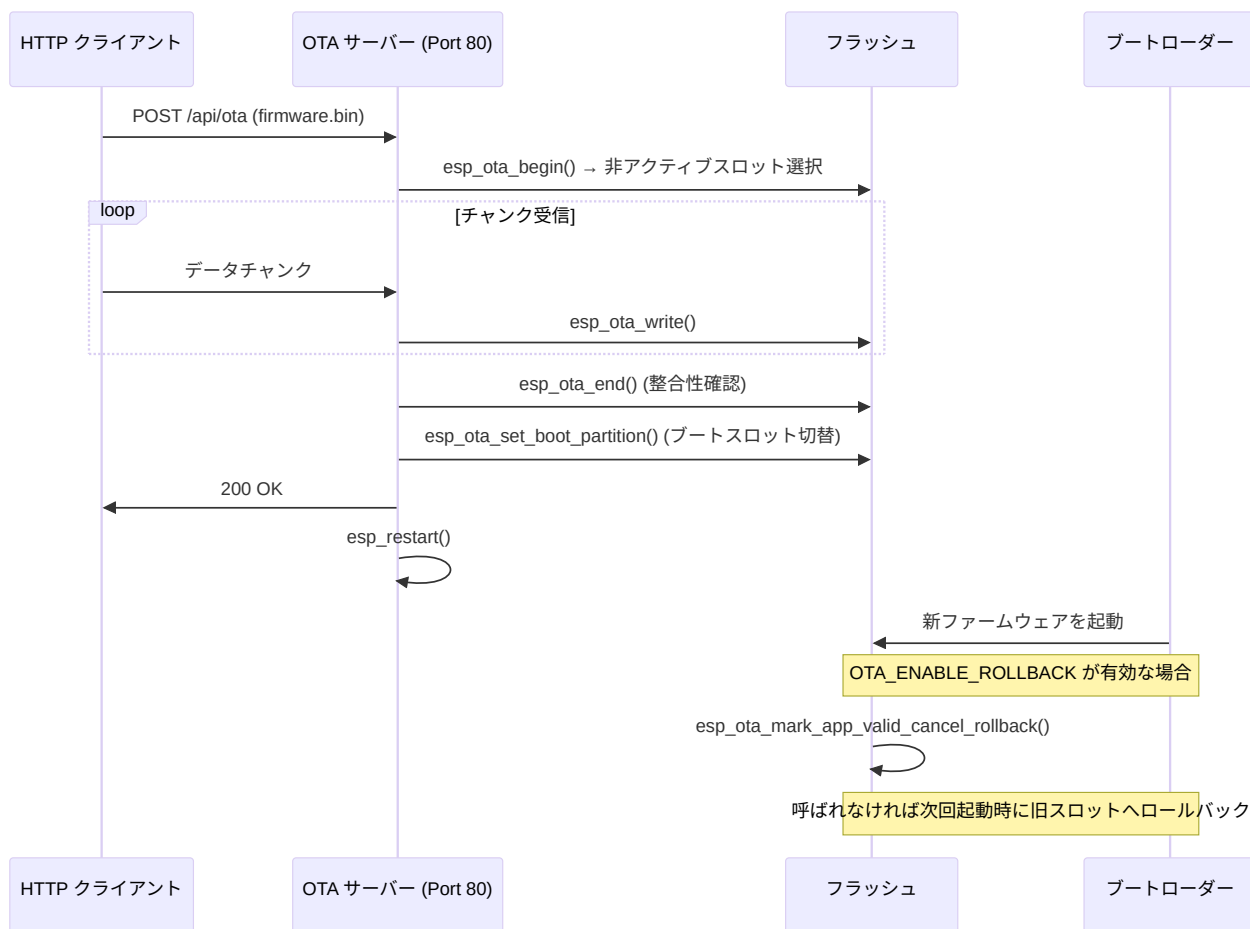
クライアントは `_serial._tcp.local` サービスを mDNS で問い合わせることで、IP アドレスを知らなくても Serial WiFi Logger のアドレスを見つけられます。TXT レコードはデバイスの接続・切断に応じて動的に更新されるので、現在のデバイス状態もここから確認できます。

第 8 章

OTA アップデート機能

ota_server.c はポート 80 で HTTP サーバーを起動し、Web UI とアップデート API を提供します。

エンドポイント	メソッド	説明
/	GET	Web UI (HTML/CSS/JS)
/api/info	GET	デバイス情報 (JSON)
/api/ota	POST	ファームウェア書き込み (バイナリ)



書き込み時は `esp_ota_begin()` でアクティブでない OTA スロットを選択し、HTTP POST ボディを逐次受信しながら `esp_ota_write()` でフラッシュに書き込みます。全データの書き込みが完了したら `esp_ota_end()` で整合性を確認し、`esp_ota_set_boot_partition()` でブートパーティションを切り替えて再起動します。

OTA_ENABLE_ROLLBACK が有効な場合、新ファームウェアは起動後に `esp_ota_mark_app_valid_cancel_rollback()` を呼び出して自分自身を「有効」とマークしなければなりません。これを行わずに再起動すると、ブートローダーが旧ファームウェアへ自動ロールバックします。

第 9 章

使用例

esptool.py による遠隔ファームウェア書き込み

Serial WiFi Logger の RFC2217 サーバーを使うと、USB ケーブルを差し替えることなく、ネットワーク越しに接続先の MCU へファームウェアを書き込めます。esptool.py は rfc2217:// 形式の URL をシリアルポートとして直接扱えるため、ほぼそのままの手順で使えます。

```
# 通常の USB 接続の場合
2 esptool.py --port /dev/ttyUSB0 --baud 2000000 \
  write_flash 0x0 firmware.bin
4
# RFC2217 経由 (Serial WiFi Logger 使用) の場合
6 esptool.py --port rfc2217://192.168.1.100:2217 --baud 2000000 \
  write_flash 0x0 firmware.bin
```

mDNS が使える環境であれば IP アドレスの代わりにホスト名を使うこともできます。

```
1 esptool.py --port "rfc2217://serial-A02048.local:2217" --baud 2000000 \
  write_flash 0x0 firmware.bin
```

DTR/RTS 経由でのリセット・ブートモード切り替えも RFC2217 の SET-CONTROL コマンドで転送されるため、esptool.py のオートリセット機能もそのまま動作します。

ESP32-S3 で USB-UART をネットワーク経由で制御する

初版 2026 年 04 月 12 日

発行	Kenta IDA
著者	Kenta IDA (@ciniml)
連絡先	fuga@fugafuga.org
Twitter	@ciniml
GitHub	https://github.com/ciniml/
